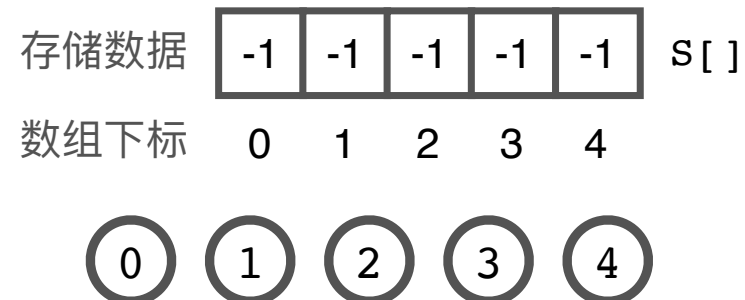




考点7 并查集的基本操作

①初始化

```
# define MAXSIZE 10
void Initial(int S[])
{
    for(int i = 0; i < MAXSIZE; i++)
        s[i] = -1;
}
```





考点7 并查集的基本操作

②查找

```
int Find(int S[], int x)
{
    while(S[x] >= 0)
        x = S[x];
    return x;
}
```

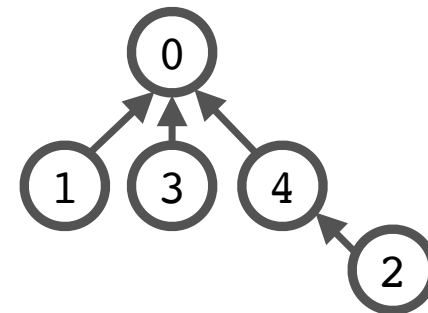
存储数据

-5	0	4	0	0
----	---	---	---	---

S[]

数组下标

0	1	2	3	4
---	---	---	---	---



Find (S, 2) = 0

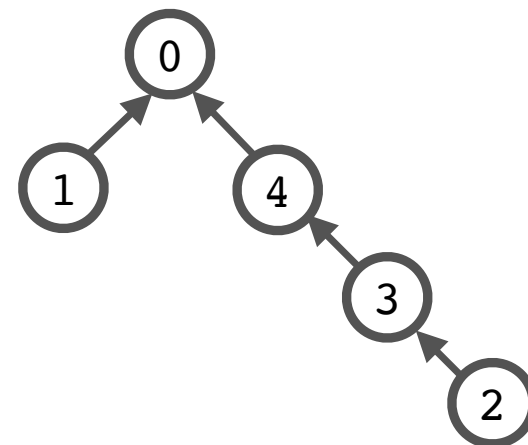


考点7 并查集的基本操作

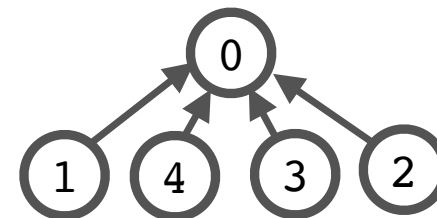
②查找

```
int FindWithPathCompression(int S[], int x)
{
    int root = x;
    while(S[root] >= 0)
        root = S[root]; //这个while是在找集合的根
    while(x != root)
    {
        int t = S[x]; //t是为了标记x的父结点
        S[x] = root; //把x挂在root下
        x = t; //接下来一直往上处理x父结点，全部挂到root下
    }
    return root;
}
```

存储数据	-5	0	3	4	0	S[]
数组下标	0	1	2	3	4	



存储数据	-5	0	0	0	0	S[]
数组下标	0	1	2	3	4	



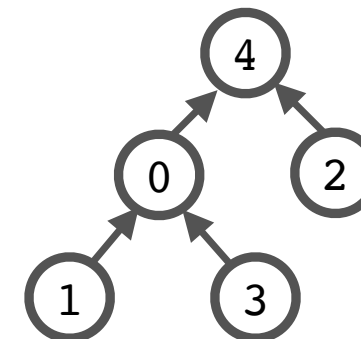
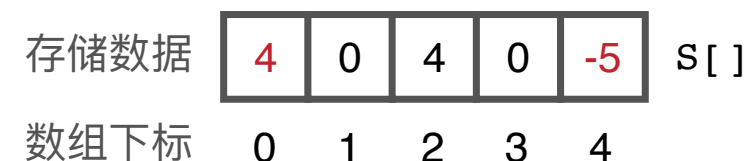
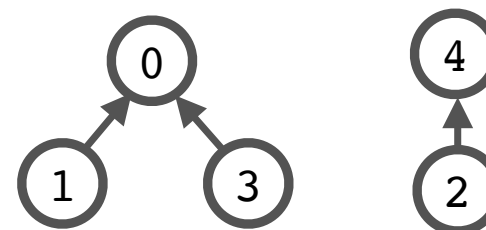
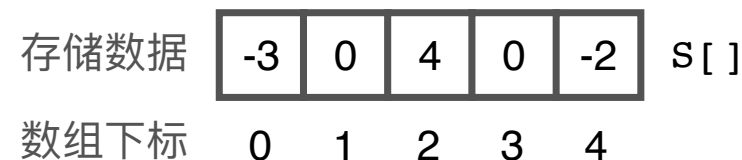
`FindWithPathCompression (S,2) = 0`



考点7 并查集的基本操作

③合并

```
void Union(int S[], int root1, int root2)
{
    if(root1 == root2) return;
    S[root1] += S[root2];
    S[root2] = root1;
}
```



Union (S, 4, 0)

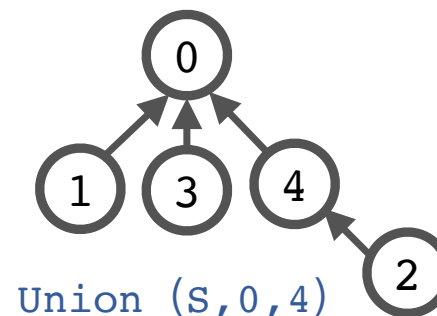
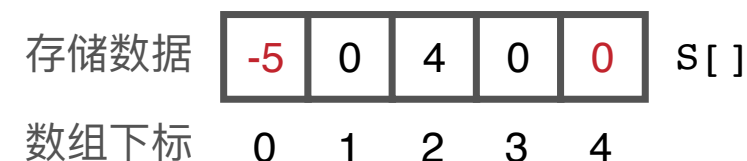
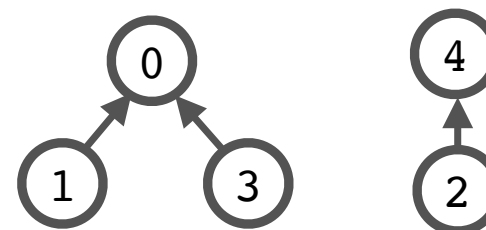
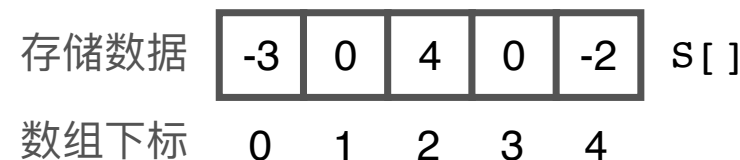


考点7 并查集的基本操作

③合并

```
void UnionBySize(int S[], int root1, int root2)
{
    if(root1 == root2)
        return;
    if(S[root1] < S[root2])
    {
        S[root1] += S[root2];
        S[root2] = root1;
    }
    else
    {
        S[root2] += S[root1];
        S[root1] = root2;
    }
}
```

优化后的Union可以将集合树的高度控制在 $O(\log_2 n)$ 以内





考点7 并查集的基本操作

使用并查集的基本操作就是先Find再Union
即 $\text{Union}(\text{Find}(x), \text{Find}(y))$

优化后的Find+Union

$O(\alpha(n))$



考点7 并查集的基本操作

④并查集的应用

检测图中是否有环

①初始化并查集

②遍历所有边:

- 对于 (u, v) :
 - Find u 所在集合的Root u
 - Find v 所在集合的Root v
- 判断是否属于同一集合:
 - 如果 $\text{Root } u = \text{Root } v$, 加入 (u, v) 会形成环
 - 如果 $\text{Root } u \neq \text{Root } v$, 使用Union合并两个集合

③完成遍历



考点7 并查集的基本操作

④并查集的应用

实现Kruskal算法

- ①初始化并查集
- ②将途中所有边按权重从小到大排序
- ③遍历所有边:
 - 对于 (u, v) :
 - Find u 所在集合的Root u
 - Find v 所在集合的Root v
 - 判断是否属于同一集合:
 - 如果 $\text{Root } u = \text{Root } v$, 跳过, 遍历下条边
 - 如果 $\text{Root } u \neq \text{Root } v$, 则添加 (u, v) 不会有环, 将这条边加入最小生成树, 使用Union操作合并两个集合
- ④如果最小生成树的边的数量已到达 $|V| - 1$, 则停止
如果边不够但没有更多的边可选择, 说明原图不连通, 无法生成MST



考点7 并查集的基本操作

④并查集的应用

判断无向图的连通性

①初始化并查集

②遍历所有边

• 对于 (u, v) :

• Find u 所在集合的Root u

• Find v 所在集合的Root v

• 判断是否属于同一集合:

• 如果 $\text{Root } u = \text{Root } v$, 什么都不做 (已连通)

• 如果 $\text{Root } u \neq \text{Root } v$, 使用Union操作合并两个集合, 这意味着 u 和 v 当前连通

③遍历图中所有顶点, 使用Find找到其根

如果所有顶点所在集合的根相同, 则图连通

考点7 并查集的基本操作

BOK数据结构强化



选择题考频: 0/16

大题考频: 0/16

习题训练

21. 关于并查集的以下说法中, 错误的是

A. 并查集可用于判断图的连通性问题

B. 使用路径压缩和按秩合并优化后, 单次操作的时间复杂度为 $O(\alpha(n))$

C. 并查集可以用来维护集合的并与交操作

D. 并查集基于双亲表示法

考点7 并查集的基本操作

BOK数据结构强化



选择题考频: 0/16

大题考频: 0/16

习题训练

22. 下列关于并查集的说法中正确的是

A. 并查集可用于实现Prim算法

B. Union操作可根据当前集合的规模, 将小集合合并到大集合中

C. 初始每个集合大小为1时, 任意合并顺序下最终形成的集合树是一棵完全二叉树

D. 路径压缩后的并查集在最坏情况下的高度是 $O(n)$





考点8 图的概念和性质

①假设有两个图 $G=(V, E)$ 和 $G'=(V', E')$, 如果 $V' \subseteq V$, $E' \subseteq E$, 则称 G' 为 G 的子图

一定要确保 V' 和 E' 是可以扯上关系的

②无向完全图有 $\frac{n(n-1)}{2}$ 条边, 有向完全图有 $n(n-1)$ 条弧

完全图一定是连通/强连通的, 并且完全图的边数是给定顶点数量时最多的

③连通分量 = 无向图中的极大连通子图, 强连通分量 = 有向图中的极大强连通子图

极大表示边数达到顶点能承受的最大数量, 即原图中和某顶点相关的边都需要出现

④生成树 = 无向连通图中一个包含图中所有顶点的极小连通子图

①极小表示保证连通的前提下边数要达到最小

②极小连通子图本身不要求必须包含图中的所有顶点

③最小生成树MST是所有生成树中边的权值之和最小的生成树

⑤对于无向图来说, 最少需要 $n-1$ 条边才能连通, 边数大于 $n-1$ 时必有环

对于有向图来说, 最少需要 n 条弧才能保证强连通 (形成一个环)

⑥无向图中所有顶点的度之和等于边数的2倍, 有向图所有顶点的入度之和=出度之和=弧数

⑦ n 个顶点的无向图最少有1个连通分量, 最多有 n 个连通分量

最少情况下 n 个顶点两两之间都有路径, 所以无向图本身就是自己唯一的连通分量

最多情况下不存在边, 每个顶点独立并自成一个连通分量



考点8 图的概念和性质

⑧【保证连通】和【可以实现连通】的区别：

①保证连通是指无论边怎么换位置，图都是连通的

②可以实现连通是指边在某些位置中可以构造出连通图，但如果换位置可能就无法确保依旧连通

【问法1】让 n 个顶点的无向图实现连通至少需要几条边？【答】 $n-1$

【问法2】让 n 个顶点的无向图保证连通至少需要几条边？【答】 $\frac{(n-1)(n-2)}{2}+1$



考点8 图的概念和性质

【2009】下列关于无向连通图特性的叙述中, 正确的是

I. 所有顶点的度之和为偶数

II. 边数大于顶点个数减1

III. 至少有一个顶点的度为1

A. 只有I

B. 只有II

C. I和II

D. I和III



考点8 图的概念和性质

【2010】若无向图 $G(V, E)$ 中含有7个顶点, 要保证图 G 在任何情况下都是连通的, 则需要的边数最少是

- A. 6
- B. 15
- C. 16**
- D. 21



考点8 图的概念和性质

【2017】已知无向图G含有16条边，其中度为4的顶点个数为3，度为3的顶点个数为4，其他顶点的度均小于3。图G所含的顶点个数至少是

A. 10

B. 11

C. 13

D. 15



考点8 图的概念和性质

【2022】对于无向图 $G=(V, E)$ ，下列选项中，正确的是

- A. 当 $|V| > |E|$ 时， G 一定是连通的
- B. 当 $|V| < |E|$ 时， G 一定是连通的
- C. 当 $|V| = |E| - 1$ 时， G 一定是不连通的
- D. 当 $|V| > |E| + 1$ 时， G 一定是不连通的**

考点8 图的概念和性质

BOK数据结构强化



选择题考频: 4/16, 分值: 2

大题考频: 0/16

习题训练

24. 若一个具有 n 个顶点、 n 条边的连通无向图中, 恰好只有一个环, 则它的不同生成树的数量是

A. 1

B. n

C. $n-1$

D. n^2

考点8 图的概念和性质

BOK数据结构强化



选择题考频：4/16，分值：2

大题考频：0/16

习题训练

25. 若 G 为含有10条边的非连通无向图，那么图 G 至少有多少个顶点？

A. 5

B. 6

C. 7

D. 8

考点8 图的概念和性质

BOK数据结构强化



选择题考频: 4/16, 分值: 2

大题考频: 0/16

习题训练

26. 具有35个顶点和10条边的无向图的连通分量最多是

A. 35

B. 30

C. 31

D. 32





考点9 邻接矩阵和邻接表

邻接矩阵

①无向图：顶点 v_i 的度 = 第 i 行元素之和 = 第 i 列元素之和

②有向图：

①顶点 v_i 的出度 = 第 i 行元素之和

②顶点 v_i 的入度 = 第 i 列元素之和

邻接表

①无向图：顶点 v_i 的度 = 第 i 个边表中的结点个数

②有向图：

①顶点 v_i 的出度 = 第 i 个边表中的结点个数

②顶点 v_i 的入度需要遍历各顶点的边表才能得出，即顶点 v_i 的入度就是在边表中出现的次数



考点9 邻接矩阵和邻接表

邻接矩阵

- ①无向图：邻接矩阵是对称的，可以压缩存储上/下三角，即需要 $\frac{n(n-1)}{2}$ 个单元来存储边
- ②有向图：n个顶点的图需要 n^2 个单元来存储边

邻接表

- ①无向图：n个顶点表结点 + 2e个边表结点（边表结点一定是偶数个）
- ②有向图：n个顶点表结点 + e个边表结点



考点9 邻接矩阵和邻接表

邻接矩阵中， $A^k[i][j]$ 表示从顶点 i 到顶点 j 之间长度为 k 的路径条数



考点9 邻接矩阵和邻接表

【2013】设图的邻接矩阵A如下所示，各顶点的度依次是

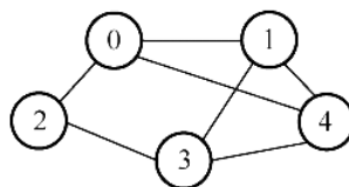
$$A = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

- A. 1, 2, 1, 2
- B. 2, 2, 1, 1
- C. 3, 4, 2, 3**
- D. 4, 4, 2, 2



考点9 邻接矩阵和邻接表

【2015】已知含有5个顶点的图G如下图所示。



请回答下列问题：

- 1) 写出图G的邻接矩阵A（行、列下标从0开始）
- 2) 求 A^2 ，矩阵 A^2 中位于0行3列元素值的含义是什么？
- 3) 若已知具有n（ $n \geq 2$ ）个顶点的图的邻接矩阵为B，则 B^m （ $2 \leq m \leq n$ ）中非零元素的含义是什么？





考点10 十字链表和邻接多重表

①结点结构

顶点结点 结点编号

data	firstin	firstout
------	---------	----------

data	用于存放数据
firstin	指向以data为弧头的第一个顶点
firstout	指向以data为弧尾的第一个顶点

顶点结点 结点编号

data	firstedge
------	-----------

data	用于存放数据
firstedge	指向第一条依附于该顶点的边

弧结点

tailvex	headvex	hlink	tlink	(info)
---------	---------	-------	-------	--------

tailvex	弧尾指向的顶点的编号
headvex	弧头指向的顶点的编号
hlink	弧头相同的下一条弧
tlink	弧尾相同的下一条弧
info	弧的信息，一般省略

边结点

ivex	ilink	jvex	jlink	(info)
------	-------	------	-------	--------

ivex	该边依附的两顶点的编号1
ilink	指向下一条依附于ivex的边
jvex	该边依附的两顶点的编号2
jlink	指向下一条依附于jvex的边
info	边的信息，一般省略



弧尾 → 弧头
tail → head

考点10 十字链表和邻接多重表

②画法

顶点结点 结点编号

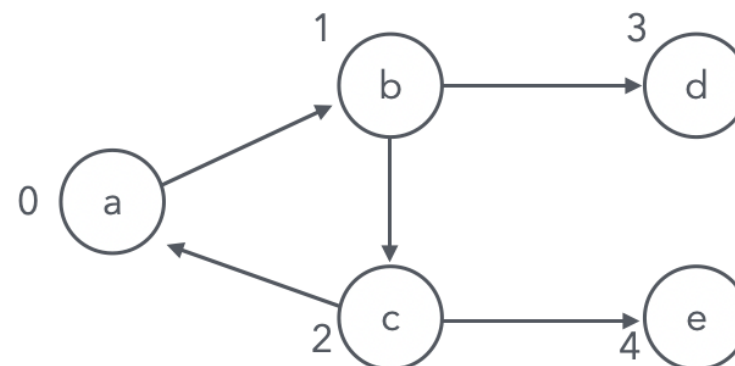
data	firstin	firstout
------	---------	----------

data	用于存放数据
firstin	指向以data为弧头的第一个顶点
firstout	指向以data为弧尾的第一个顶点

弧结点

tailvex	headvex	hlink	tlink	(info)
---------	---------	-------	-------	--------

tailvex	弧尾指向的顶点的编号
headvex	弧头指向的顶点的编号
hlink	弧头相同的下一条弧
tlink	弧尾相同的下一条弧
info	弧的信息, 一般省略





考点10 十字链表和邻接多重表

②画法

顶点结点

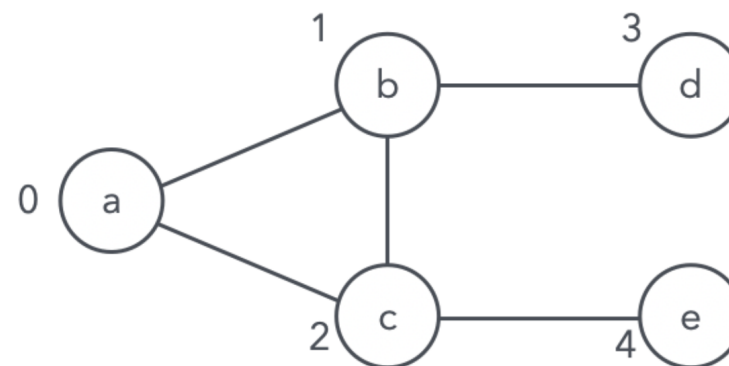
结点编号	data	firstedge
------	------	-----------

data	用于存放数据
firstedge	指向第一条依附于该顶点的边

边结点

ivex	ilink	qvex	jlink	(info)
------	-------	------	-------	--------

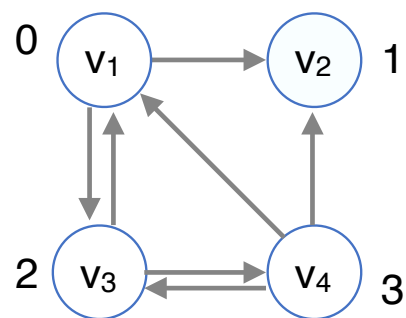
ivex	该边依附的两顶点的编号1
ilink	指向下一条依附于ivex的边
qvex	该边依附的两顶点的编号2
jlink	指向下一条依附于qvex的边
info	边的信息，一般省略



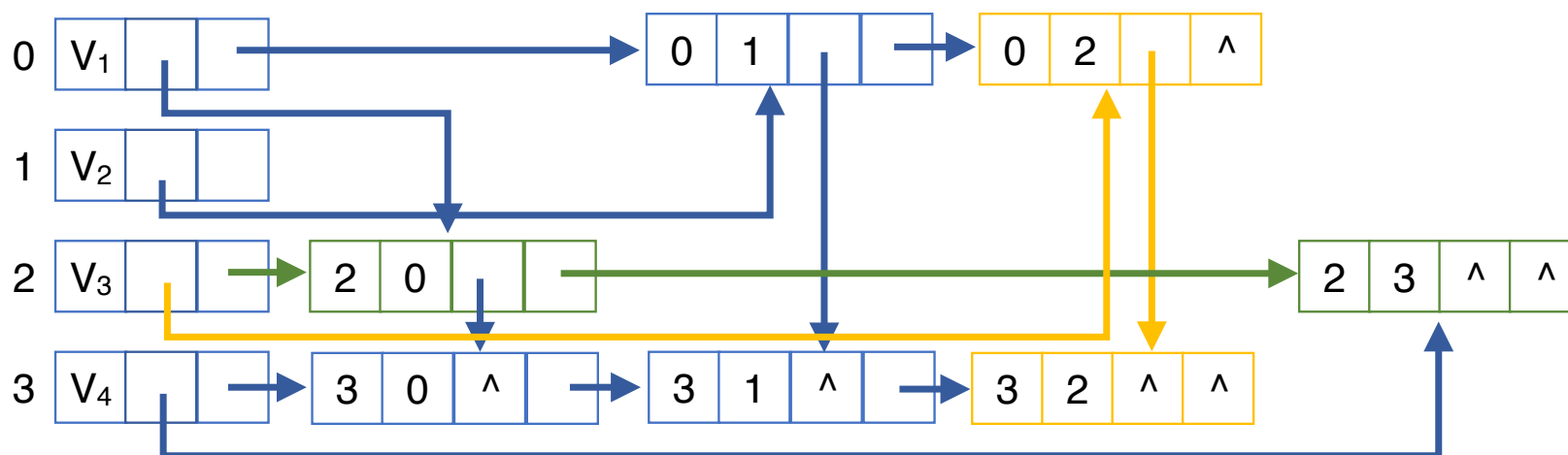


考点10 十字链表和邻接多重表

③优势



有向图G

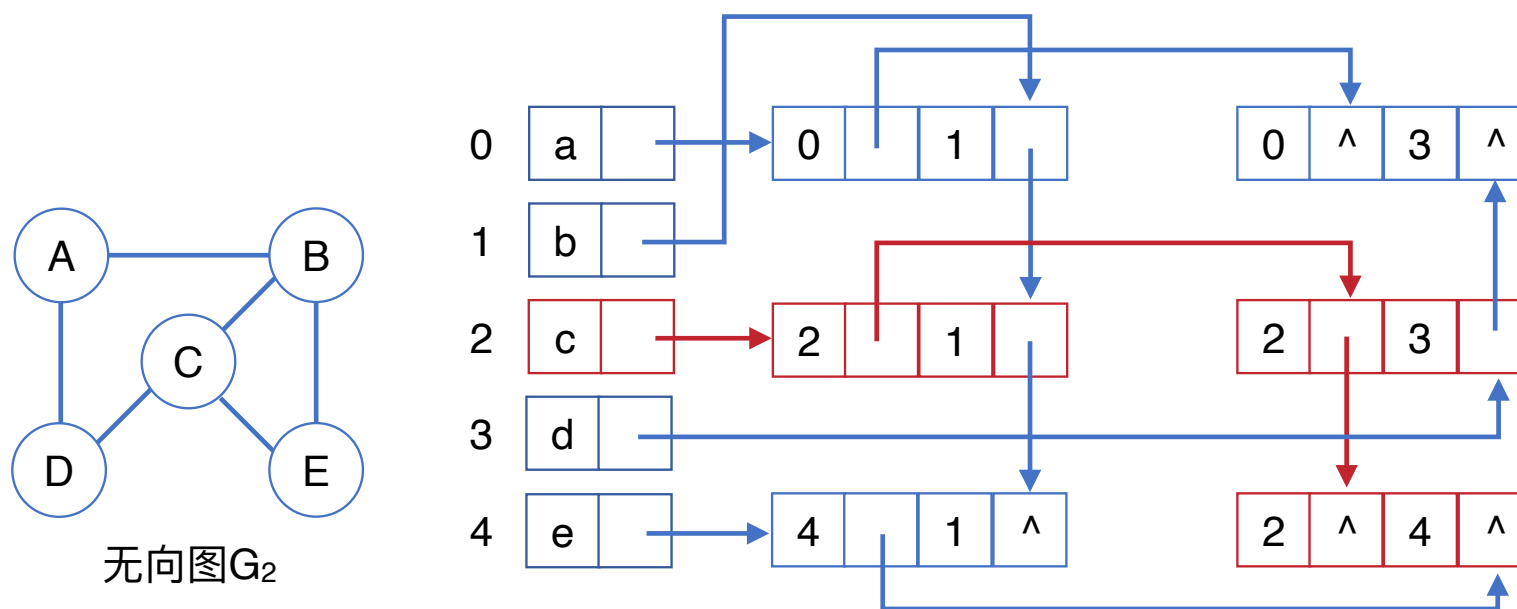


十字链表结合了邻接表和逆邻接表，可以方便地计算顶点 v_i 的入度和出度/寻找依附于某一结点的所有弧



考点10 十字链表和邻接多重表

③优势

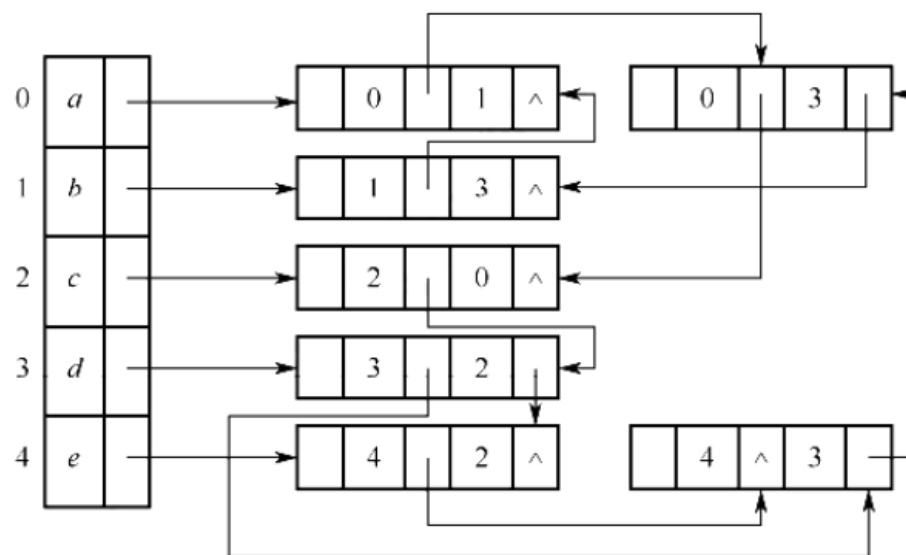


邻接多重表中，每条边只会被存储一次，且所有依附于同一结点的边都被串联在一个链表中，便于**计算结点的度**/寻找依附于某一结点的所有边



考点10 十字链表和邻接多重表

【2024】若无向图 $G=(V, E)$ 的邻接多重表如下图所示, 则 G 中顶点 b 与 d 的度分别是



A. 0, 2

B. 2, 4

C. 2, 5

D. 3, 4





考点11 DFS和BFS

①DFS思想及在不同存储结构上的实现

```
bool visited[MVNum];  
void DFSTraverse(Graph G)  
{  
    for(v = 0; v < G.vexnum; ++v)  
        visited[v] = false;  
    for(v = 0; v < G.vexnum; ++v)  
        if(!visited[v]) DFS(G,v);  
}
```

```
void DFS(Graph G, int v)  
{  
    cout << v;  
    visited[v] = true;  
    for(w = FirstAdjVex(G,v); w >= 0; w = NextAdjVex(G,v,w))  
    {  
        if(!visited[w])  
            DFS(G,w);  
    }  
}
```

DFSTraverse()中调用DFS()的次数 = 连通分量的个数

从一个顶点开始，先标记访问并输出它，再去检查它的邻接点
如果邻接点未被访问过，就对邻接点再次调用DFS
重复上述操作，直到所有顶点都被标记访问



考点11 DFS和BFS

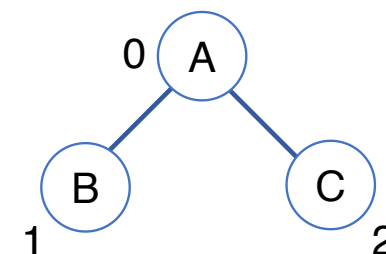
①DFS思想及在不同存储结构上的实现

DFS核心语句

```
for(w = FirstAdjVex(G,v); w >= 0; w = NextAdjVex(G,v,w))  
{  
    if(!visited[w])  
        DFS(G,w);  
}
```

从给定的参数顶点 v 开始,
首先遍历邻接矩阵中第 v 行的所有元素,
如果第 w 列的值不为 0, 说明顶点 v 与顶点 w 之间存在边
若此时顶点 w 尚未被访问, 则将 w 作为参数递归调用 DFS
继续以同样方式访问 w 的所有邻接点
直到所有可达顶点都被访问为止

时间复杂度为 $O(n^2)$



图G

0	1	1
1	0	0
1	0	0

对应邻接矩阵



考点11 DFS和BFS

①DFS思想及在不同存储结构上的实现

DFS核心语句

```
for(w = FirstAdjVex(G,v); w >= 0; w = NextAdjVex(G,v,w))
{
    if(!visited[w])
        DFS(G,w);
}
```

从给定的参数顶点 v 开始

先访问 v 所对应的邻接表, 获取其第一个邻接点指针 p

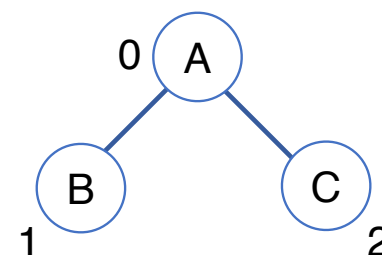
然后沿着邻接表的边表不断向后遍历,

每次取出当前边所指向的邻接点 w ,

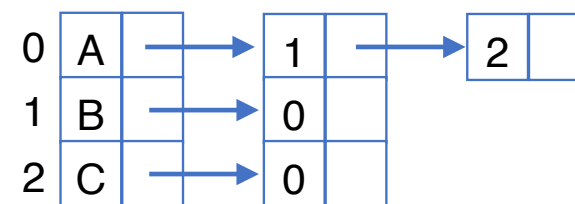
如果 w 尚未被访问, 就将 w 作为参数递归调用 DFS

直到所有可达顶点都被访问为止

时间复杂度为 $O(n+e)$



图G

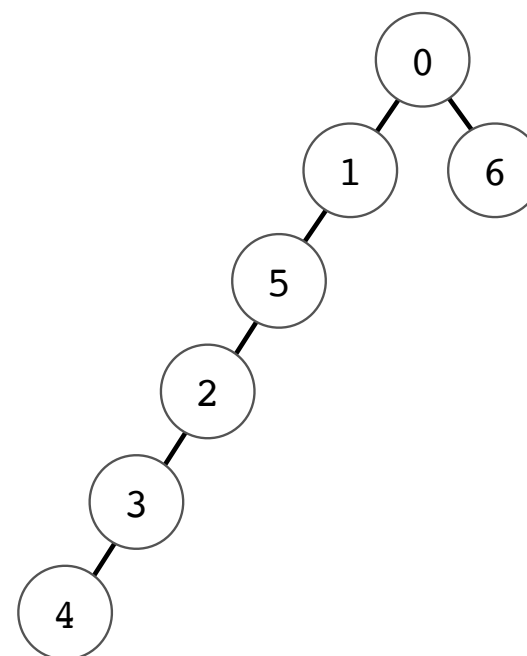
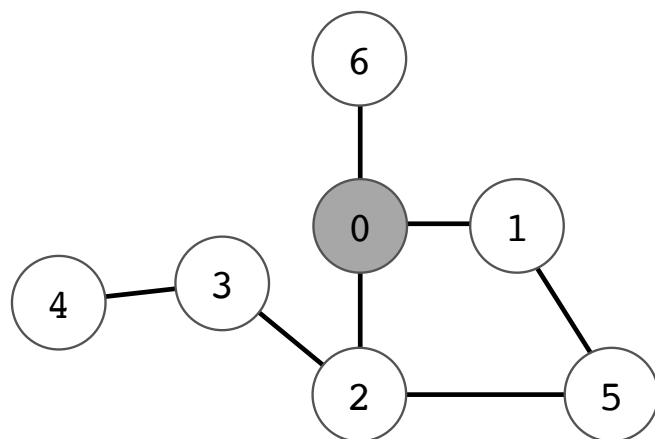


对应邻接表



考点11 DFS和BFS

①DFS思想及在不同存储结构上的实现





考点11 DFS和BFS

②BFS思想及在不同存储结构上的实现

```
bool visited[MVNum];  
void BFSTraverse(Graph G)  
{  
    for(v = 0; v < G.vexnum; ++v)  
        visited[v] = false;  
    for(v = 0; v < G.vexnum; ++v)  
        if(!visited[v]) BFS(G,v);  
}
```

BFSTraverse()中调用BFS()的次数 = 连通分量的个数

从一个顶点开始
先标记访问并输出它, 再将它加入队列
然后不断从队列中取出队头顶点
检查该顶点的所有邻接点
如果某个邻接点未被访问过
就标记访问、输出, 并加入队列
重复上述过程, 直到队列为空为止

```
void BFS(Graph G, int v)  
{  
    cout << v;  
    visited[v] = true;  
    InitQueue(Q);  
    EnQueue(Q,v);  
    while(!QueueEmpty(Q))  
        (Q,u);  
    for(w = FirstAdjVex(G,u); w >= 0; w = NextAdjVex(G,u,w))  
        if(!visited[w])  
        {  
            cout << w;  
            visited[w] = true;  
            EnQueue(Q,w);  
        }  
}
```




考点11 DFS和BFS

②BFS思想及在不同存储结构上的实现

从给定的参数顶点 v 开始, 首先输出顶点 v , 并将其标记为已访问, 将顶点 v 入队,

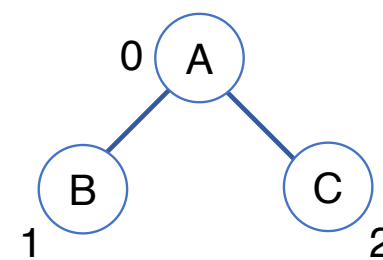
当队列不为空时循环执行以下操作:

❶从队列中取出队头顶点 u , 遍历邻接矩阵第 u 行的所有元素, 依次检查每个顶点 w

❷如果 $G.arcs[u][w]$ 不为 0, 说明 u 与 w 之间存在边

❸如果顶点 w 尚未被访问, 则输出 w 、标记访问, 并将 w 入队, 继续检查 u 行中剩下的所有列, 直到扫描完整行

时间复杂度为 $O(n^2)$



图G

0	1	1
1	0	0
1	0	0

对应邻接矩阵



辅助队列



考点11 DFS和BFS

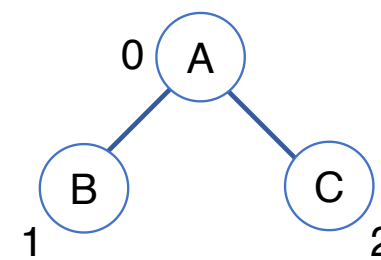
②BFS思想及在不同存储结构上的实现

从给定的参数顶点 v 开始, 首先输出顶点 v , 并将其标记为已访问
将顶点 v 入队

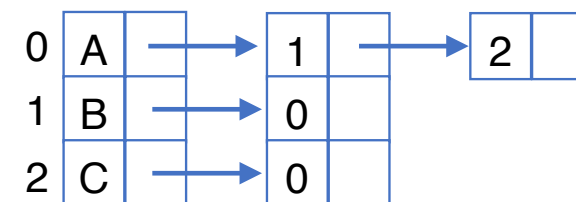
当队列不为空时循环执行以下操作:

- ❶从队列中取出队头顶点 u , 访问顶点 u 所对应的邻接表, 获取第一个邻接点指针 p
- ❷沿着邻接表不断向后遍历, 每次取出当前边指向的邻接点 w
- ❸如果 w 尚未被访问, 则输出 w 并标记访问, 并将 w 入队
- ❹继续遍历邻接表的下一项, 直到 p 为 `NULL`

时间复杂度为 $O(n+e)$



图G



对应邻接表

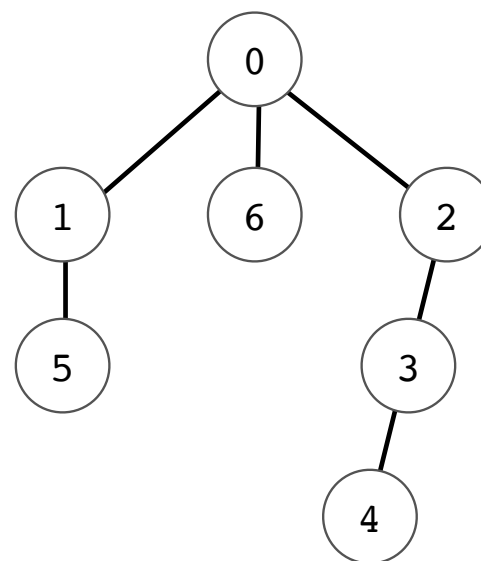
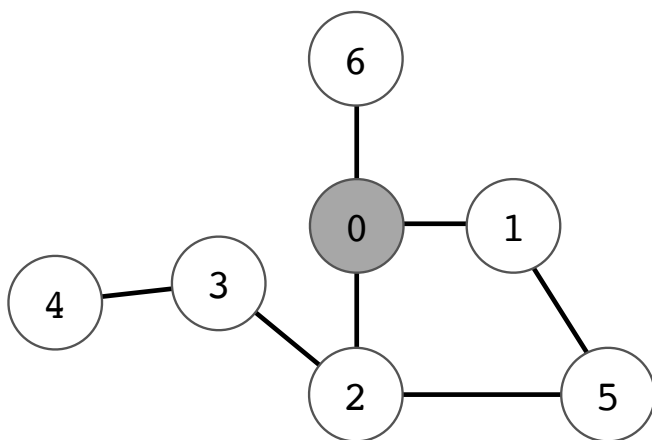


辅助队列



考点11 DFS和BFS

②BFS思想及在不同存储结构上的实现

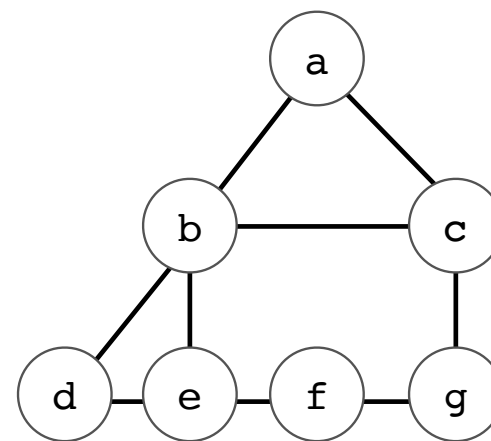
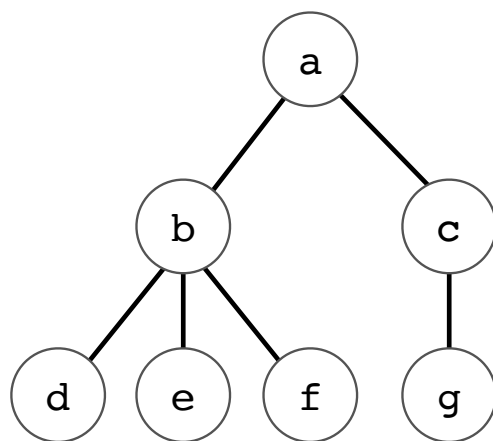




考点11 DFS和BFS

③图与树的遍历关联

图的DFS类似于树的先序遍历
图的BFS类似于树的层序遍历





考点11 DFS和BFS

④DFS和BFS的应用

DFS	BFS
判断环路	(权值相同的有权图/无权图) 单源最短路径
拓扑排序	
判断图是否连通	

通过对代码进行修改，BFS实际上也可以用于判断环路或实现拓扑排序
但DFS的思路更符合这两个应用场景，所以更常用



考点11 DFS和BFS

④DFS和BFS的应用

DFS判断有向图是否存在环路

- ❶从某个顶点出发进行DFS
- ❷在遍历过程中, 如果遇到某个邻接点处于正在访问的状态, 说明该有向图存在环路
- ❸为了准确判断, 通常需要为每个顶点维护三种状态: 未访问、正在访问、已访问

```
void CheckGraphCycle(Graph G)
{
    for(v = 0; v < G.vexnum; v++)
    {
        if(status[v] == Unvisited)
            DetectCycleDFS(G, v);
    }

    if(hasCycle)
        cout << "图中存在回路";
    else
        cout << "图中无回路";
}
```

```
bool hasCycle = false;
void DetectCycleDFS(Graph G, int v)
{
    status[v] = Visiting; // 正在访问
    for(w = FirstAdjVex(G, v); w >= 0; w = NextAdjVex(G, v, w))
    {
        if(status[w] == Visiting) // 再次遇到正在访问的点 → 环路
            hasCycle = true;
        if(status[w] == Unvisited)
            DetectCycleDFS(G, w);
    }
    status[v] = Visited; // 已访问
}
```



考点11 DFS和BFS

④DFS和BFS的应用

DFS判断无向图是否存在环路

- ❶从某个顶点出发进行DFS
- ❷在遍历过程中, 如果遇到某个邻接点处于正在访问的状态, 并且这个正在访问的节点不是上一步访问的节点, 说明该无向图存在环路
- ❸为了准确判断, 通常需要为每个顶点维护三种状态: 未访问、正在访问、已访问

```
void CheckGraphCycle(Graph G)
{
    for(v = 0; v < G.vexnum; v++)
    {
        if(status[v] == Unvisited)
            DetectCycleDFS(G, v, -1);
        // 初始结点没有父节点
    }

    if(hasCycle)
        cout << "图中存在回路";
    else
        cout << "图中无回路";
}
```

```
bool hasCycle = false;
void DetectCycleDFS(Graph G, int v, int parent)
{
    status[v] = Visiting;
    for(w = FirstAdjVex(G, v); w >= 0; w = NextAdjVex(G, v, w))
    {
        if(status[w] == Visiting && w != parent)
            hasCycle = true;
        if(status[w] == Unvisited)
            DetectCycleDFS(G, w, v); // 传递当前节点作为w的父节点
    }
    status[v] = Visited;
}
```

**DFS实现拓扑排序**

- ①每个顶点v在DFS退出递归前都会记录当前时间戳time, 并赋值给 finishTime[v]
- ②因为后继节点的DFS会先于v完成, 因此它们的finishTime一定比v小
- ③最后按照finishTime从大到小排序所有顶点, 就得到了一个合法的拓扑序列

考点11 DFS和BFS

④DFS和BFS的应用

```
bool visited[MVNum];
int finishTime[MVNum];
int time;
void TopoSortDFS(Graph G, int v)
{
    visited[v] = true;
    for(w = FirstNeighbor(G, v); w >= 0; w = NextNeighbor(G, v, w))
    {
        if(!visited[w])
            TopoSortDFS(G, w);
    }
    time = time + 1;
    finishTime[v] = time;
}
```




BFS计算单源最短路径

- ❶从起点 v 出发，将其距离初始化为0，其余所有顶点初始化为 ∞ ，并将 v 入队作为搜索起点
- ❷每次从队列中取出顶点 u ，访问它的所有未访问邻接点 w ，并将 $d[w]$ 设为 $d[u]+1$
- ❸因为BFS一层层推进的，第一个访问到的路径一定最短，最终 $d[]$ 数组记录的即为起点到每个顶点的最短路径长度

考点11 DFS和BFS

④DFS和BFS的应用

```
void BFS_DISTANCE(Graph G, int u)
{
    for(i = 0; i < G.vexnum; i++)
        d[i] =  $\infty$ ;
    visited[u] = TRUE;
    d[u] = 0;
    EnQueue(Q, u);
    while(!isEmpty(Q))
    {
        DeQueue(Q, u);
        for(w = FirstNeighbor(G, u); w >= 0; w = NextNeighbor(G, u, w))
        {
            if(!visited[w])
            {
                visited[w] = TRUE;
                d[w] = d[u] + 1;
                EnQueue(Q, w);
            }
        }
    }
}
```



考点11 DFS和BFS

④DFS和BFS的应用

```
bool visited[MVNum];
void DFSTraverse(Graph G)
{
    for(v = 0; v < G.vexnum; ++v)
        visited[v] = false;
    DFS(G,0);
    for(v = 0; v < G.vexnum; ++v)
        if(!visited[v])
            return false;
    return true;
}
```

```
bool visited[MVNum];
void BFSTraverse(Graph G)
{
    for(v = 0; v < G.vexnum; ++v)
        visited[v] = false;
    BFS(G,0);
    for(v = 0; v < G.vexnum; ++v)
        if(!visited[v])
            return false;
    return true;
}
```

DFS/BFS判断图是否连通

图是连通的 $\Leftrightarrow$ 从任意一个顶点出发可以访问到所有其他顶点



考点11 DFS和BFS

【2012】对有 n 个顶点、 e 条边且使用邻接表存储的有向图进行广度优先遍历，其算法的时间复杂度是

A. $O(n)$

B. $O(e)$

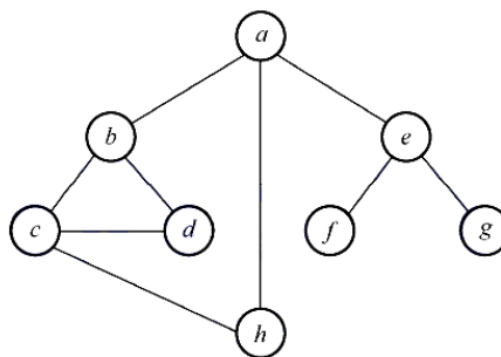
C. $O(n+e)$

D. $O(ne)$



考点11 DFS和BFS

【2013】下列选项中，不是如下无向图的广度优先遍历序列的是



A. h, c, a, b, d, e, g, f

B. e, a, f, g, b, h, c, d

C. d, b, c, a, h, e, f, g

D. a, b, c, d, h, e, f, g



考点11 DFS和BFS

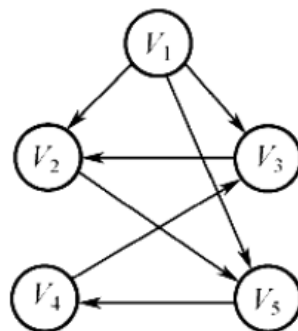
【2015】设有向图 $G = (V, E)$ ，顶点集 $V = \{V_0, V_1, V_2, V_3\}$ ，边集 $E = \{<V_0, V_1>, <V_0, V_2>, <V_0, V_3>, <V_1, V_3>\}$ 。若从顶点 V_0 开始对图进行深度优先遍历，则可能得到的不同遍历序列个数是

- A. 2
- B. 3
- C. 4
- D. 5**



考点11 DFS和BFS

【2016】下列选项中，不是下图深度优先搜索序列的是



- A. V_1, V_5, V_4, V_3, V_2
- B. V_1, V_3, V_2, V_5, V_4
- C. V_1, V_2, V_5, V_4, V_3
- D. V_1, V_2, V_3, V_4, V_5



考点11 DFS和BFS

【2020】修改递归方式实现的图的深度优先搜索（DFS）算法，将输出（访问）顶点信息的语句移到退出递归前（执行输出语句后立刻退出递归）。采用修改后的算法遍历有向无环图G，若输出结构中包含G中的全部顶点，则输出的顶点序列是G的

- A. 拓扑有序序列
- B. 逆拓扑有序序列**
- C. 广度优先搜索序列
- D. 深度优先搜索序列

```
void DFS(Graph G, int v)
{
    for(w = FirstAdjVex(G,v); w >= 0; w = NextAdjVex(G,v,w))
    {
        if(!visited[w])
            DFS(G,w);
    }
    cout << v;
    visited[v] = true;
}
```



考点11 DFS和BFS

【2023】已知无向连通图G中各边的权值均为1。在下列算法中，一定能够求出图G中从某顶点到其余各顶点最短路径的是

I. Prim算法

II. Kruskal算法

III. 图的广度优先搜索算法

A. 仅I

B. 仅III

C. 仅I、II

D. I、II、III

考点11 DFS和BFS

BOK数据结构强化



选择题考频: 6/16, 分值: 2

大题考频: 0/16

习题训练

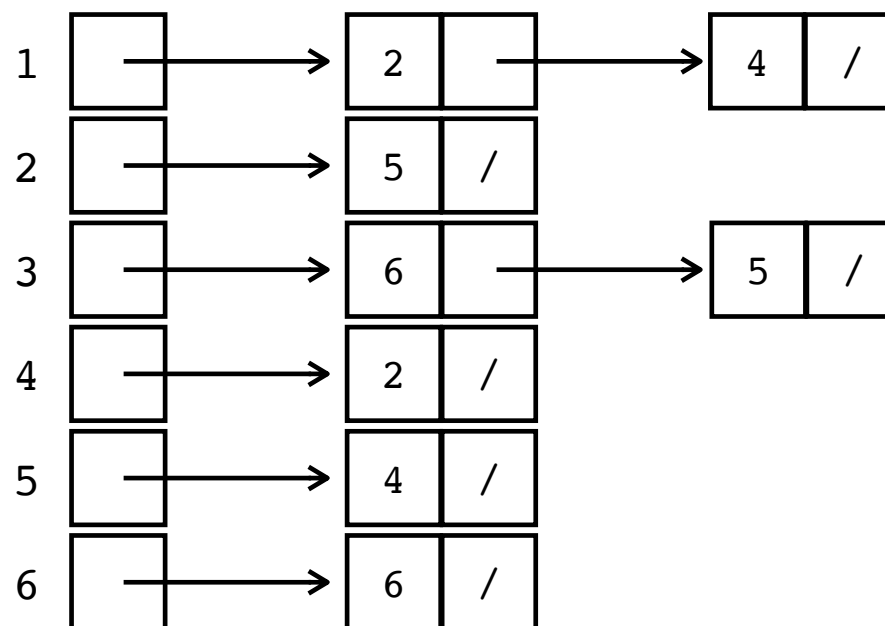
27. 一个有向图G的邻接表存储如图所示, 若从顶点1开始进行DFS, 可以得到的DFS序列是

A. 125436

B. 124536

C. 124563

D. 362514



考点11 DFS和BFS

BOK数据结构强化



选择题考频：6/16，分值：2

大题考频：0/16

习题训练

28. 在如下邻接矩阵表示的无向图中，以顶点a为起点进行广度优先遍历（BFS），下列哪一个序列是遍历序列的结果？

$$\begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 1 \\ 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

A. a, b, c, d, e

B. a, c, b, d, e

C. a, c, d, b, e

D. a, b, c, e, d





考点12 拓扑排序

有向无环图

1. 不能出现重复的操作数顶点
2. 尽可能化简

$$(x+y) * ((x+y) / x)$$



考点12 拓扑排序

拓扑排序是对有向无环图的顶点的一种排序，它使得若存在一条从顶点A到顶点B的路径，则在排序中B出现在A的后面

1.在有向图中选择一个入度为0的顶点并输出它

2.从图中删除该顶点和它的所有出度

重复这2步操作，直到不存在入度为0的顶点

若最终输出的顶点数小于有向图中的顶点数，则图中**存在环**

否则输出的顶点序列即为该图的一个拓扑序列



考点12 拓扑排序

- ①拓扑排序可能不唯一
- ②若有向图中有环，则该图不存在拓扑排序
- ③若有向图的拓扑有序序列唯一，则图中入度为0和出度为0的顶点都只有1个
- ④顶点数大于1的强连通图不能进行拓扑排序
- ⑤如果拓扑序列中A在B之前，原图中不一定有 $\langle A, B \rangle$
- ⑥有向无环图的拓扑序列唯一，并不能唯一确定该图



考点12 拓扑排序

【2019】用有向无环图描述表达式 $(x+y)((x+y)/x)$, 需要的顶点个数至少是

A. 5

B. 6

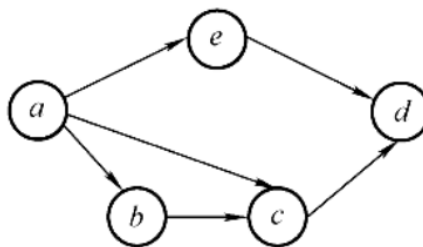
C. 8

D. 9



考点12 拓扑排序

【2010】对下图进行拓扑排序，可得不同拓扑序列的个数是



A. 4

B. 3

C. 2

D. 1



考点12 拓扑排序

【2012】若用邻接矩阵存储有向图，矩阵中主对角线以下的元素均为零，则关于该图拓扑排序的结论是

- A. 存在，且唯一
- B. 存在，且不唯一
- C. 存在，可能不唯一**
- D. 无法确定是否存在



考点12 拓扑排序

【2011】下列关于图的叙述中，正确的是

I. 回路是简单路径

II. 存储稀疏图，用邻接矩阵比邻接表更省空间

III. 若有向图中存在拓扑序列，则该图不存在回路

A. 仅II

B. 仅I、II

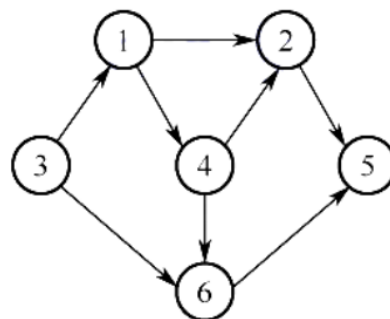
C. 仅III

D. 仅I、III



考点12 拓扑排序

【2014】对下图所示的有向图进行拓扑排序，得到的拓扑序列可能是



A. 3, 1, 2, 4, 5, 6

B. 3, 1, 2, 4, 6, 5

C. 3, 1, 4, 2, 5, 6

D. 3, 1, 4, 2, 6, 5



考点12 拓扑排序

【2016】若对 n 个顶点、 e 条弧的有向图采用邻接表存储，则拓扑排序算法的时间复杂度是

A. $O(n)$

B. $O(n+e)$

C. $O(n^2)$

D. $O(ne)$

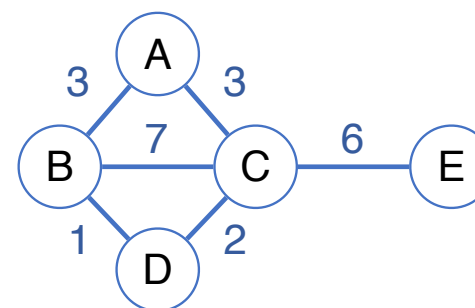
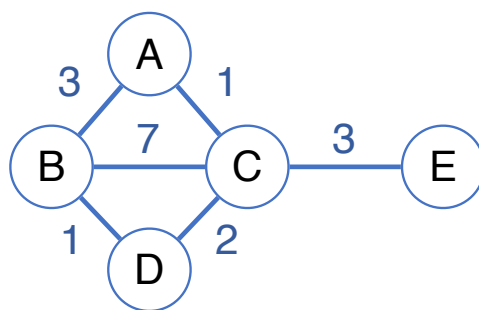




考点13 最小生成树

生成树	最小生成树MST
无向连通图中一个包含图中所有顶点的极小连通子图	所有生成树中边的权值之和最小的生成树

- ①只有无向连通图才有生成树的概念
- ② n 个顶点的图, 最小生成树有 $n-1$ 条边
- ③对于最小生成树, 再加一条边就会形成环路, 再减一条边就会不连通
- ④若原图存在权值相同的边, 那么该图的最小生成树**可能**不唯一
- ⑤若原图的边权值互不相等, 那么该图的最小生成树**一定**唯一





考点13 最小生成树

Prim算法	Kruskal算法
选点	选边
适用于边稠密，顶点少图	适用于边稀疏，顶点多的图
①初始时任取一顶点加入树T ②选择一个与当前T中顶点集合距离最近的顶点并将其顶点和相应边加入T 每次操作后T的顶点数和边数都加一 ③直到所有顶点加入T，此时有n-1条边	①初始时为只有n个顶点而无边的非连通图T，每个顶点自成一个连通分量 ②按照边的权值从小到大的顺序，选择当前未被选取且权值最小的边。若该边依附的顶点落在T中不同的连通分量上，则将此边加入T，否则舍弃此边而选择下一条 ③直到T中所有顶点都在一个连通分量上
$O(V ^2)$	$O(E \log_2 E)$



考点13 最小生成树

【2012】下列关于最小生成树的叙述中, 正确的是

I. 最小生成树的代价唯一

II. 所有权值最小的边一定会出现在所有的最小生成树中

III. 使用Prim算法从不同顶点开始得到的最小生成树一定相同

IV. 使用Prim算法和Kruskal算法得到的最小生成树总不相同

A. 仅I

B. 仅II

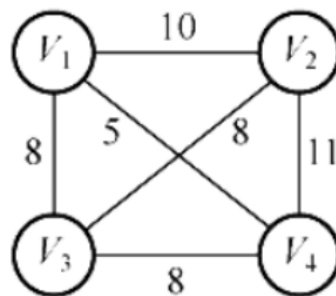
C. 仅I、III

D. 仅II、IV



考点13 最小生成树

【2015】求下面的带权图的最小（代价）生成树时，可能是Kruskal算法第2次选中但不是Prim算法（从 v_4 开始）第2次选中的边是

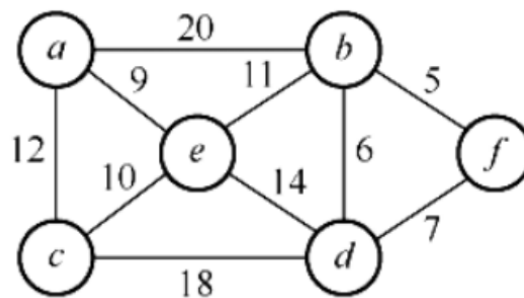


- A. (V_1, V_3)
- B. (V_1, V_4)
- C. (V_2, V_3)**
- D. (V_3, V_4)



考点13 最小生成树

【2020】已知无向图G如下所示, 使用Kruskal算法求图G的最小生成树, 加到最小生成树中的边依次是



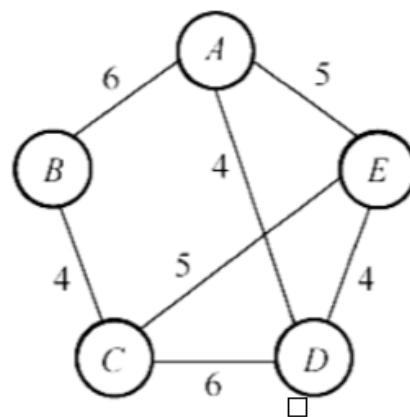
- A. (b, f), (b, d), (a, e), (c, e), (b, e)
- B. (b, f), (b, d), (b, e), (a, e), (c, e)
- C. (a, e), (b, e), (c, e), (b, d), (b, f)
- D. (a, e), (c, e), (b, e), (b, f), (b, d)



考点13 最小生成树

【2017】使用Prim算法求带权连通图的最小（代价）生成树（MST）。请回答下列问题：

- 1) 对下列图G，从顶点A开始求G的MST，依次给出按算法选出的边。
- 2) 图G的MST是唯一的吗？
- 3) 对任意的带权连通图，满足什么条件时，其MST是唯一的？



考点13 最小生成树

BOK数据结构强化



选择题考频：4/16，分值：2

大题考频：2/16，分值：8~12

习题训练

29. 下列关于最小生成树的叙述中错误的是

- A. 无向连通图中任何一个边数最少且连通所有顶点的子图都是该图的生成树
- B. 用不同算法求最小生成树得到的结果都是一样的**
- C. 某图的最小生成树中，某条边的权值可能大于另一条未入选的边的权值
- D. n 个顶点的最小生成树只能有 $n-1$ 条边

考点13 最小生成树

BOK数据结构强化



选择题考频：4/16，分值：2

大题考频：2/16，分值：8~12

习题训练

30. 已知无向图G的邻接矩阵如下（无边表示为 ∞ ），若从顶点a开始执行Prim算法构造最小生成树，则以下哪组边是Prim算法的选边顺序？

	a	b	c	d	e
a	0	2	∞	6	∞
b	2	0	3	8	5
c	∞	3	0	∞	7
d	6	8	∞	0	9
e	∞	5	7	9	0

A. a-b b-c b-e a-d

B. a-b b-e e-c a-d

C. a-b b-c c-e e-d

D. a-b b-c c-e c-d





考点14 最短路径

①Dijkstra

<code>final[]</code>	标注各顶点是否已找到最短路径
<code>dist[]</code>	最短路径长度
<code>path[]</code>	路径上的前驱

①初始化三个数组信息

②观察`final[]`数组, 若`final[i]=0`且`dist[i]`最小, 则将`final[i]`改为1, 收录该顶点 v_i 到最优路径集合中

③对 v_i 的邻接点 v_j 做如下判断:

如果 $\text{dist}[i] + \langle v_i, v_j \rangle \text{的权值} < \text{dist}[j]$, 则更新`dist[j]`与`path[j]`

④循环②和③, 直到所有顶点都找到最短路径

Dijkstra用于求解单源最短路径, 不适用于带负权值的图

Dijkstra算法的时间复杂度为 $O(|V|^2)$



考点14 最短路径

②Floyd

- ①初始化矩阵
- ②依次选择顶点作为中转点
- ③以中转点为中心，画出横、竖、对角线三条彩带，彩带上的数字不变，逐个检查彩带外的每个数字，若该数字所在行、列对应的彩带上的值相加小于数字本身，则用更小的值替换，否则保持不变
- ④循环②和③，直到找到所有顶点对之间的最短路径

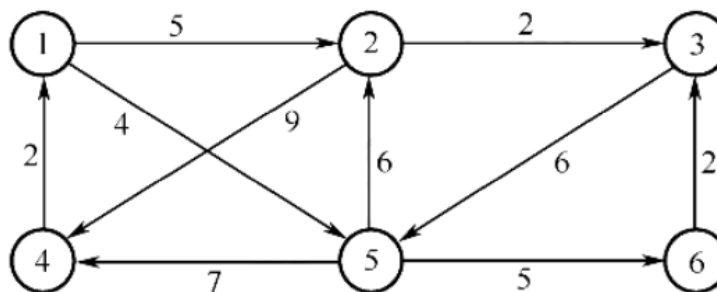
Floyd用于求解各顶点之间的最短路径，不适用于带负权回路的图

Floyd算法的时间复杂度为 $O(|V|^3)$



考点14 最短路径

【2016】使用Dijkstra算法求下图中从顶点1到其他各顶点的最短路径，依次得到的各最短路径的目标顶点是



- A. 5, 2, 3, 4, 6
- B. 5, 2, 3, 6, 4**
- C. 5, 2, 4, 3, 6
- D. 5, 2, 6, 3, 4

考点14 最短路径

BOK数据结构强化



选择题考频: 3/16, 分值: 2

大题考频: 2/16, 分值: 10

习题训练

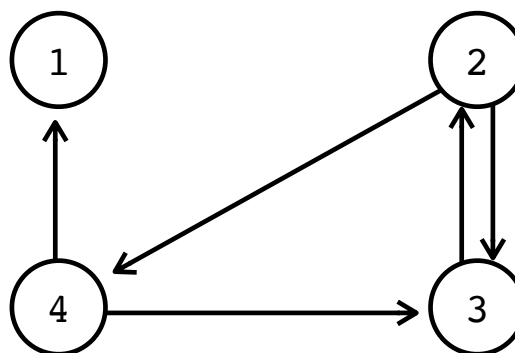
31. 用Floyd算法求下图的最短路径, 得到的dist矩阵结果是

A.
$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

B.
$$\begin{bmatrix} 1 & \infty & \infty & 0 \\ 2 & 1 & 1 & 1 \\ 3 & 1 & 1 & 2 \\ 1 & 2 & 1 & 1 \end{bmatrix}$$

C.
$$\begin{bmatrix} 0 & 0 & 0 & 0 \\ 2 & 0 & 1 & 1 \\ 3 & 1 & 0 & 2 \\ 1 & 2 & 1 & 0 \end{bmatrix}$$

D.
$$\begin{bmatrix} 0 & \infty & \infty & \infty \\ 2 & 0 & 1 & 1 \\ 3 & 1 & 0 & 2 \\ 1 & 2 & 1 & 0 \end{bmatrix}$$







考点15 关键路径

事件 v_i 的最早发生时间 v_e	$v_e(\text{源点})=0$ $v_e(k) = \max\{v_e(j) + \text{Weight}(V_j, V_k)\}$ v_k 为 v_j 的任意后继
事件 v_i 的最迟发生时间 v_l	$v_l(\text{汇点})=v_e(\text{汇点})$ $v_l(k) = \min\{v_l(j) - \text{Weight}(V_k, V_j)\}$ v_k 为 v_j 的任意前驱
活动 a_i 的最早开始时间 e	该弧起点顶点的 v_e
活动 a_i 的最迟开始时间 l	该弧终点的顶点的 v_l 减去弧的权值
活动的时间余量 $=l-e$ 关键活动的时间余量为0 ($l=e$) , 关键活动组成了关键路径	



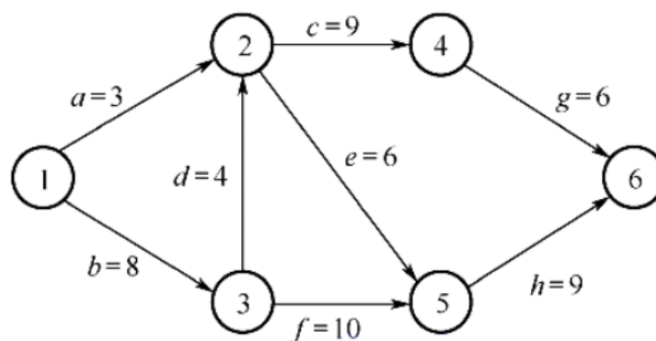
考点15 关键路径

- ①活动的时间余量= $1-e$
- ②关键活动的时间余量为0 ($1=e$) , 关键活动组成了关键路径
- ③关键路径可能不唯一
- ④关键路径是持续时间最长 (权值之和最大) 的那条路径
- ⑤缩短关键路径, 并不一定导致整个工期加快
 - ❶如果只有一条关键路径, 缩短关键路径, 一定导致整个工期加快
 - ❷如果有多条关键路径, 缩短关键路径, 不一定导致整个工期加快, 因为缩短了路径长度的那条关键路径将不再是关键路径
 - ❸缩短所有关键路径上共有的任意一个关键活动的持续时间才会导致整个工期加快
 - ❹延长任意一条关键路径上的关键活动的持续时间都会导致整个工期减慢
- ⑥求关键路径是以拓扑排序为基础的



考点15 关键路径

【2013】下列AOE网表示一项包含8个活动的工程。通过同时加快若干活动的进度可缩短整个工程的工期。在下列选项中，加快其进度就可缩短工程工期的是



A. c和e

B. d和c

C. f和d

D. f和h



考点15 关键路径

【2020】若使用AOE网估算工程进度，则下列叙述中正确的是

- A. 关键路径是从源点到汇点边数最多的一条路径
- B. 关键路径是从源点到汇点路径长度最长的路径**
- C. 增加任意一个关键活动的时间不会延长工程的工期
- D. 缩短任意一个关键活动的时间将会缩短工程的工期



考点15 关键路径

【2011】已知有6个顶点（顶点编号为0~5）的有向带权图G，其邻接矩阵A为上三角矩阵，按行为主序（行优先）保存在如下的一维数组中。

4	6	∞	∞	∞	5	∞	∞	∞	4	3	∞	∞	3	3
---	---	----------	----------	----------	---	----------	----------	----------	---	---	----------	----------	---	---

要求：

- 1) 写出图G的邻接矩阵A。
- 2) 画出有向带权图G。
- 3) 求图G的关键路径，并计算该关键路径的长度。



祝你今天特别特别开心^•u-^